Document number: P0652R2
Project: Programming Language C++
Audience: SG1 Concurrency

Sergey Murylev, Yandex Ltd, <SergeyMurylev@gmail.com>, <smurylev@yandex-team.ru>
Anton Malakhov, Intel Corp., <anton.malakhov@intel.com>
Antony Polukhin, Yandex.Taxi Ltd, <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2019-01-15

Concurrent associative data structure with unsynchronized view

Significant changes since P0652R2 are marked with blue.

I. Introduction and Motivation

There's a lot of use-cases where a concurrent modification of unordered associative container is required. For example, there is a popular web service named Memcached which simply caches objects in memory. Also, we can use it to collect statistics, to store connection metadata or just use as lightweight key-value storage. Some big companies like Facebook have it in their standard library. There were attempts to add such containers/data structures in the past (N3425, N3732, and others...)

This paper is another attempt to deal with the problem. This time we are trying to keep the interface familiar to users, hard to misuse but still functional.

Reference implementation is available at https://github.com/BlazingPhoenix/concurrent-hash-map.

II. Design decisions

A. Allow Open Addressing:

When users wish to use the concurrent associative data structure, they are searching for performance and scalability. Fastest known implementations rely on the **open addressing** MemC3.pdf, so the interface of this proposal allows having Open Addressing implementation under the hood.

B. No functions that are easy to misuse:

In C++17 `std::shared_ptr::use_count()` function was removed because users were misusing it. Users were hoping that the result of the function is actual at the point where they were trying to use it. However, as soon as the result is returned from the function it could expire as someone modifies the value from other thread.

We followed the C++17 decision and **removed all the functions that are useless/dangerous** in concurrent environments: `size()`, `count()`, `empty()`, `buckets_count()` and so forth.

C. No iterators:

Iterators must take care of synchronization, otherwise, the user can not dereference the iterator at all. If Iterators do some synchronization it **affects performance**. Otherwise, if Iterators do some synchronization then **deadlocks will happen**. For example, if in first thread we iterate from beginning to the end of the container and in the second thread we iterate from the end to the beginning, then the deadlock will happen in the middle as the iterators met.

It is possible to drop the open addressing idea and make the iterators to have shared ownership of buckets. In that case, iterator may keep the bucket alive. This seems implementable and usable by users but significantly **affects performance** by adding multiple additional atomic operations and adding additional indirections. We tried to stick to this idea for a long time and minimize the performance impact. Finally, we created a list of use-cases for concurrent associative data structures and found out that in all of those use-cases iterators are useless or kill performance of the whole class (so are also useless). Instead of that, we came up with an idea of **unsynchronized view/policy**.

D. View/policy with a full interface:

This is the **killer feature** of this proposal that attempts to fix all the limitations from above and provide a much more useful interface.

The idea is following: multiple operations on unordered containers make sense only if that container is not concurrently modified. A user may take the responsibility that no-one is modifying the container at this point and gain all the operations and iterators.

Such approach allows to initialize/prepare the data for container **without additional synchronization overhead.** It also **allows advanced usage**:

- Multiple threads use the same `concurrent_unordered_map` for reads and write simultaneously.
- Multiple threads use `const unordered_map_view` on the same `concurrent_unordered_map` simultaneously (no modifications through the `concurrent_unordered_map` interface are allowed!).
- The single thread uses `unordered_map_view` (no modifications are allowed from other threads!).
- Multiple threads use the same `const concurrent_unordered_map&` for reads and multiple threads use `const unordered_map_view` on the same concurrent_unordered_map simultaneously (ineffective: use multiple `const unordered_map_view` instead).
- A user can select whether or not to lock the whole container when it creates a view. It can be useful if we want to suspend concurrent usage, to take for example snapshot of contents and serialize state to some storage. If the user decided to use view without lock it's the user responsibility to make sure that there is no concurrent access form other threads, otherwise, it leads to undefined behavior.

E. No node operations, different from `std::unordered_map` iterator invalidation:

This is a consequence of allowing the open addressing as an underlying implementation.

F. Allow element visitation using the synchronization of the container:

This is a very risky decision because it unleashes new ways for deadlocking/breaking the container (users may recursively visit the same value; users may call heavy functions that will degrade the overall performance of the container; users can call some parallel functions of the standard library that may potentially use the same mutexes as the container implementation...).

However, there's a lot of use-cases where a value must be updated depending on the value that is in the container. Without a visitation, there's no way to do that safely, as all the functions return values by copy. See examples B and C.

G. Allow to visit the contents of the container:

We've added an ability to visit all elements of the container without locking the whole container. In this case, contents can be occasionally changed during iteration. Such functionality can be useful for debugging.

H. We don't propose smart pointer like interface to container items:

Some implementations (for example Intel TBB) provide a possibility to get an item from a container with an appropriate lock guard. There are two possible ways to implement it:

1. We have a lock for each item. In this case, we dramatically increase the size of the container if we use it to store small types, and also we add extra CPU effort to lock each item. If we store as `mapped_type` some lock guarded type (for example `boost::synchronized_value`) we can get thread-safe shared ownership of the collection objects without a risk of getting deadlocks.
2. We have a lock for some group of items (bucket). In this case, we can get a deadlock if we try to obtain smart pointer like objects for two items from the same bucket and use them simultaneously.

According to thoughts provided above, we considered not to propose such interface because it decreases performance and creates a very simple ability for a user to make a deadlock.

III. Draft interface and informal description

???.1 Header <concurrent_unordered_map>

```
namespace std {
  template <class Key,
            class T,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Allocator = allocator<pair<const Key, T>> >
  class concurrent_unordered_map;

  template <class Key,
            class T,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Allocator = allocator<pair<const Key, T>> >
  void swap(concurrent_unordered_map< Key, T, Hash, Pred, Allocator>& lhs,concurrent_unordered_map< Key, T, Hash, Pred, Allocator>& rhs);
}
```

???.2 Class `concurrent_unordered_map`

```
namespace std {
  template <class Key,
            class T,
            class Hash = hash<Key>,
            class Pred = equal_to<Key>,
            class Allocator = allocator<pair<const Key, T>> >
  class concurrent_unordered_map {
  public:
    // type aliases
    using key_type          = Key;
    using mapped_type       = T;
    using value_type        = pair<const Key, T>;
    using hasher            = Hash;
    using key_equal         = Pred;
```

```
    using allocator_type    = Allocator;
    using size_type         = implementation-defined;

    class unordered_map_view;

    // construct/copy/assign/destroy
    concurrent_unordered_map();
    explicit concurrent_unordered_map(size_type n);
    concurrent_unordered_map(size_type n, const Hash& hf,
                                    const key_equal& eql = key_equal(),
                                    const allocator_type& a = allocator_type());
    template <typename InputIterator>
    concurrent_unordered_map(InputIterator first, InputIterator last,
                             size_type n = implementation-defined,
                             const hasher& hf = hasher(),
                             const key_equal& eql = key_equal(),
                             const allocator_type& a = allocator_type());

    concurrent_unordered_map(const Allocator&);
    concurrent_unordered_map(initializer_list<value_type> il,
                             size_type n = implementation-defined,
                             const hasher& hf = hasher(),
                             const key_equal& eql = key_equal(),
                             const allocator_type& a = allocator_type());

    concurrent_unordered_map(concurrent_unordered_map&& other) noexcept;
    concurrent_unordered_map(concurrent_unordered_map&& other, const Allocator&);

    concurrent_unordered_map& operator=(concurrent_unordered_map&& other) noexcept;
    concurrent_unordered_map& operator=(initializer_list<value_type>il);

    ~concurrent_unordered_map();


    // members observers
    allocator_type get_allocator() const;
    hasher hash_function() const;
    key_equal key_eq() const;


    // visitation
    template <typename Visitor>
    void visit(const key_type& key, Visitor& f);
    template <typename Visitor>
    void visit(const key_type& key, Visitor& f) const;

    template <typename Visitor>
    void visit_all(Visitor& f);
    template <typename Visitor>
    void visit_all(Visitor& f) const;

    template<typename K, typename Visitor, typename... Args>
    bool visit_or_emplace(K&& key, Visitor& f, Args&&... args);


    // access
    optional<mapped_type> find(const key_type& key) const;
    template<typename... Args>
    mapped_type find(const key_type& key, Args&&... args) const;


    // modifiers
    template<typename K, typename... Args>
    bool emplace(K&& key, Args&&... args);
    template<typename K, typename... Args>
    bool insert_or_assign(K&& key, Args&&... args);
    template<typename... Args>
    size_type update(const key_type& key, Args&&... args);
    size_type erase(const key_type& key);

    template<class H2, class P2>
    void merge(concurrent_unordered_map<Key, T, H2, P2, Allocator>& source);
    template<class H2, class P2>
    void merge(concurrent_unordered_map<Key, T, H2, P2, Allocator>&& source);

    void swap(concurrent_unordered_map&)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
            is_nothrow_swappable_v<Hash> &&
            is_nothrow_swappable_v<Pred>);
    void clear() noexcept;


    // view retrieval
    unordered_map_view make_unordered_map_view(bool lock = false) const noexcept;
};
```

The `concurrent_unordered_map` class is an associative data structure that provides an effective key-value storage. Concurrent member functions call on the same instance of `concurrent_unordered_map` do not introduce data races.

`key_type` shall satisfy *Cpp17MoveConstructible* requirements.

`mapped_type` shall satisfy *Cpp17MoveConstructible* requirements.

Unless specified otherwise all the member functions of `concurrent_unordered_map` have the following additional requirements, and remarks:

> *Requires:* Concurrent member function invocations for the same instance of `Hash`, `Pred`, and `Allocator` do not introduce data races. Concurrent invocations for different instances of `Key` and `T` do not introduce data races for the following functions: all the constructors (including default, move and copy constructors); copy and move assignments; destructor.

Remarks: Call to member function do not introduce data races with other calls to the same or different member functions for the same or different instances of `concurrent_unordered_map`.

Calls to functions that successfully modify keys or values synchronize with calls to functions successfully reading the value or key of the same keys.

Destructor call for the `unordered_map_view` referencing the instance of `concurrent_unordered_map` synchronize with calls to functions successfully reading the value or key of the same instance of `concurrent_unordered_map`.

???.2.1 concurrent_unordered_map construct/copy/assign/destroy

```
concurrent_unordered_map();
explicit concurrent_unordered_map(size_type n);
concurrent_unordered_map(size_type n, const Hash& hf,
                                 const key_equal& eql = key_equal(),
                                 const allocator_type& a = allocator_type());
template <typename InputIterator>
concurrent_unordered_map(InputIterator first, InputIterator last,
                         size_type n = implementation-defined,
                         const hasher& hf = hasher(),
                         const key_equal& eql = key_equal(),
                         const allocator_type& a = allocator_type());

concurrent_unordered_map(const Allocator&);
concurrent_unordered_map(initializer_list<value_type> il,
                         size_type n = implementation-defined,
                         const hasher& hf = hasher(),
                         const key_equal& eql = key_equal(),
                         const allocator_type& a = allocator_type());
```

Effects: Constructs `concurrent_unordered_map` with the analogous to the `unoredered_map` effects.

```
concurrent_unordered_map(concurrent_unordered_map&& other) noexcept;
concurrent_unordered_map(concurrent_unordered_map&& other, const Allocator&);
```

Effects: Constructs `concurrent_unordered_map` with the content of `other`, leaving other in valid but unspecified state.

Remarks: `other` before the constructor call may not be equal to `*this` after the constructor call only in case of concurrent access to the `other`.

```
concurrent_unordered_map& operator=(concurrent_unordered_map&& other) noexcept;
```

Effects: Assigns the content of `other` to `*this`, leaving `other` in valid but unspecified state.

Remarks: `other` before the operator call may not be equal to `*this` only in case of concurrent access to the `other` or concurrent access to `*this`.

```
concurrent_unordered_map& operator=(initializer_list<value_type>il);
```

Effects: Assigns the content of `li` to `*this`.

Remarks: `li` before the operator call may not be equal to `*this` only in case of concurrent access to `*this`.

```
~concurrent_unordered_map();
```

Remarks: Invocation of this function concurrently with other member functions of the same instance may introduce data races.

???.2.2 concurrent_unordered_map member observers

```
allocator_type get_allocator() const;
hasher hash_function() const;
key_equal key_eq() const;
```

Expects: Copy construction of `allocator_type`, `hasher` and `key_equal` should not introduce data races.

Returns: Copies of `allocator_type`, `hasher` and `key_equal` respectively.

???.2.3 concurrent_unordered_map visitation

```
template <typename Visitor>
void visit(const key_type& key, Visitor& f);
```

Constraints: `is_invocable_v<Visitor, mapped_type&>`

Effects: Invokes `f` on the value stored with a key equivalent to the `key`.

Remarks: Modifications of the value in `f` do not introduce data races.

```
template <typename Visitor>
void visit(const key_type& key, Visitor& f) const;
```

Constraints: `is_invocable_v<Visitor, const mapped_type&>`

Effects: Invokes `f` on the value stored with a key equivalent to the `key`.

Remarks: Reads of the value passed into `f` do not introduce data races.

```
template <typename Visitor>
void visit_all(Visitor& f);
```

Constraints: `is_invocable_v<Visitor, const key_type&, mapped_type&>`

Effects: Sequentially invokes `f` on key and value pairs stored in `*this`.

Remarks: Reads or modifications of non-const arguments passed into `f` do not introduce data races. It is guaranteed to visit all the keys of `*this` only if there is no concurrent modifications of `*this`. *[Note:* Invocation of `f` on some key and value does not prevent modifications of other keys and values in `*this`. - *end note]*.

```
template <typename Visitor>
void visit_all(Visitor& f) const;
```

Constraints: `is_invocable_v<Visitor, const key_type&, const mapped_type&>`

Effects: Sequentially invokes `f` on key and value pairs stored in `*this`.

Remarks: Reads of the arguments passed into `f` do not introduce data races. It is guaranteed to visit all the keys of `*this` only if there is no concurrent modifications of `*this`. *[Note:* Invocation of `f` on some key and value does not prevent modifications of other keys and values in `*this`. - *end note]*.

```
template<typename K, typename Visitor, typename... Args>
bool visit_or_emplace(K&& key, Visitor& f, Args&&... args);
```

Constraints: `is_invocable_v<Visitor, mapped_type&> && is_constructible_v<key_type, K&&> && is_constructible_v<mapped_type, Args&&...>`

Effects: Invokes `f` on value stored with the key equivalent to `key` if such key exist in `*this`. Otherwise stores `key_type(std::forward<Key>(key))` and `mapped_type(std::forward<Args>(args)...)`.

Remarks: Construction of `key_type` and `mapped_type`, access or modification of the arguments passed into `f` do not introduce data races.

???.2.4 concurrent_unordered_map access

```
optional<mapped_type> find(const key_type& key) const;
```

Constraints: `is_copy_constructible_v<mapped_type>`

Returns: Empty optional if there is no key equivalent to `key` in `*this`, otherwise returns an optional holding a copy of mapped_type for that key.

```
template<typename... Args>
mapped_type find(const key_type& key, Args&&... args) const;
```

Constraints: `is_copy_constructible_v<mapped_type> && is_constructible_v<mapped_type, Args&&...>`

Returns: `mapped_type(std::forward<Args>(args...))` if there is no key equivalent to `key` in `*this`, otherwise returns a copy of mapped_type for that key.

???.2.5 concurrent_unordered_map modifiers

```
template<typename K, typename... Args>
bool emplace(K&& key, Args&&... args);
```

Constraints: `is_constructible_v<key_type, K&&> && is_constructible_v<mapped_type, Args&&...>`

Returns: `true` if key `key` was not in the container and function emplaced it, `false` otherwise

```
template<typename K, typename... Args>
bool insert_or_assign(K&& key, Args&&... args);
```

Constraints: `is_constructible_v<key_type, K&&> && is_constructible_v<mapped_type, Args&&...> && is_move_assigneble_v<mapped_type>`

Returns: `true` if key `key` was not in the container and function emplaced it, `false` if the key was in container and mapped_type for that key was replaced by move assigning `mapped_type(std::forward<Args>(args...))`

```
template<typename... Args>
size_type update(const key_type& key, Args&&... args);
```

Constraints: `is_constructible_v<mapped_type, Args&&...> && is_move_assigneble_v<mapped_type>`

Returns: `0` if the `key` was not in the container, `1` if the key was in the container and mapped_type for that key was replaced by move assigning `mapped_type(std::forward<Args>(args...))`

```
size_type erase(const key_type& key);
```

Returns: `0` if the `key` was not in the container, `1` if the key was in the container and was erased.

```
template<class H2, class P2>
void merge(concurrent_unordered_map<Key, T, H2, P2, Allocator>& source);
template<class H2, class P2>
void merge(concurrent_unordered_map<Key, T, H2, P2, Allocator>&& source);
```

Effects: Merges the content of `source` into `*this`. For each key and value pair from `source` apply the following rules: If `*this` already has the key from `source`, key and value are left in the `source`. Otherwise, key and value are moved into `*this`. *[Note:* If new values are being concurrently inserted into `source` during this operation then at the end of this function invocation `source` may have keys that do not exist in `*this`- *end note]*

```
void swap(concurrent_unordered_map& other)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
            is_nothrow_swappable_v<Hash> &&
            is_nothrow_swappable_v<Pred>);
```

Effects: Swaps the content of `other` into `*this`. *[Note:* If new values are being concurrently inserted into `other` or `*this` during this operation then at the end of this function invocation `other` may not be equal to `*this` before the invocation and at the end of this function invocation `*this` may not be equal to `other` before the invocation- *end note]*

Remarks: Does not invoke any move, copy, or swap operations on individual elements.

```
void clear() noexcept;
```

Effects: Clears the content of `*this`. *[Note:* If new values are being concurrently inserted into `*this` during this operation then at the end of this function invocation `*this` may contain some keys- *end note]*

???.2.6 concurrent_unordered_map view retrieval

```
unordered_map_view make_unordered_map_view(bool lock = false) const noexcept;
```

Effects: Creates a view of the container contents. If the argument is `true` any concurrent access to `*this` through member functions or any subsequent attempt to invoke `this->make_unordered_map_view(true)` should block as long as the view is not destroyed.

Synchronization: Synchronizes with previous modifications of `*this`.

???.3 Class `unordered_map_view`

```
  template <class Key, class T, class Hash, class Pred, class Allocator>
    class  concurrent_unordered_map<Key, T, Hash, Pred, Allocator>::unordered_map_view {
    concurrent_unordered_map<Key, T, Hash, Pred, Allocator>& delegate; // exposition only

  public:
    // types:
    using key_type          = Key;
    using mapped_type       = T;
    using value_type        = pair<const Key, T>;
    using hasher            = Hash;
    using key_equal         = Pred;
    using allocator_type    = Allocator;

    using pointer           = typename allocator_traits<Allocator>::pointer;
    using const_pointer     = typename allocator_traits<Allocator>::const_pointer;
    using reference         = value_type&;
    using reference         = const value_type&;
    using size_type         = implementation-defined;
    using difference_type   = implementation-defined;
    using iterator          = implementation-defined;
    using const_iterator    = implementation-defined;
    using local_iterator    = implementation-defined;
    using const_local_iterator = implementation-defined;

    // construct/copy/destroy
    unordered_map_view() = delete;
    unordered_map_view(unordered_map_view&&) noexcept = delete;
    unordered_map_view(const unordered_map_view&) noexcept = delete;
    unordered_map_view& operator=(unordered_map_view&&) noexcept = delete;
    unordered_map_view& operator=(const unordered_map_view&) noexcept = delete;
    ~unordered_map_view();

    // iterators:
    iterator        begin() noexcept;
    const_iterator  begin() const noexcept;
    iterator        end() noexcept;
    const_iterator  end() const noexcept;
    const_iterator  cbegin() const noexcept;
    const_iterator  cend() const noexcept;

    // capacity:
    bool empty() const noexcept;
    size_type size() const noexcept;
    size_type max_size() const noexcept;

    // modifiers:
    template<typename... Args>
    pair<iterator, bool> emplace(Args&&... args);
    // We considered have only forwarding reference variant of insert to avoid interface overloading
    template<class P> pair<iterator, bool> insert(P&& x);
    template<class InputIterator>
    void insert(InputIterator first, InputIterator last);
    void insert(initializer_list<value_type> il);

    template <typename... Args>
      pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
```

```
    template <typename... Args>
      pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
    template <class M>
      pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
    template <class M>
      pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);

    iterator erase(iterator position);
    iterator erase(const_iterator position);
    size_type erase(const key_type& k);
    iterator erase(const_iterator first, const_iterator last);
    void swap(concurrent_unordered_map&)
        noexcept(allocator_traits<Allocator>::is_always_equal::value &&
            is_nothrow_swappable_v<Hash> &&
            is_nothrow_swappable_v<Pred>);
    void clear() noexcept;

    template<class H2, class P2>
    void merge(concurrent_unordered_map<Key, T, H2, P2, Allocator>&& source);
    template<class H2, class P2>
    void merge(concurrent_unordered_multimap<Key, T, H2, P2, Allocator>&& source);

    // observers:
    allocator_type get_allocator() const;
    hasher hash_function() const;
    key_equal key_eq() const;

    // map operations:
    iterator find(const key_type& k);
    const_iterator find(const key_type& k) const;
    size_type count(const key_type& k) const;
    pair<iterator, iterator> equal_range(const key_type& k);
    pair<const_iterator, const_iterator> equal_range(const key_type& k) const;

    // element access:
    mapped_type& operator[](const key_type& k);
    mapped_type& operator[](key_type&& k);
    const mapped_type& at(const key_type& k) const;
    mapped_type& at(const key_type& k);

    // bucket interface:
    size_type bucket_count() const;
    size_type max_bucket_count() const;
    size_type bucket_size(size_type n);
    size_type bucket(const key_type& k) const;
    local_iterator begin(size_type n);
    const_local_iterator begin(size_type n) const;
    local_iterator end(size_type n);
    const_local_iterator end(size_type n) const;
    const_local_iterator cbegin(size_type n) const;
    const_local_iterator cend(size_type n) const;

    // hash policy:
    void rehash(size_type n);
    float load_factor() const noexcept;
  };
}
```

`unordered_map_view` class refers `concurrent_unordered_map` and provides an interface to `concurrent_unordered_map` that satisfies unordered associative container requirements [unord.req] except iterator invalidation requirements, `load_factor` functions, `size()` complexity requirements, buckets and node operations.

[*Note:* Concurrent const member functions calls on the instances of `unordered_map_view` referencing the same `concurrent_unordered_map` introduce data races - *end note.*]

[*Note:* Concurrent member functions calls on the `concurrent_unordered_map` instance *A* and on the `unordered_map_view` referencing the instance *A* introduce data races. - *end note.*]

???.3.1 unordered_map_view construct/copy/assign/destroy

`~unordered_map_view();`

> *Effects:* If `*this` was created by `make_unordered_map_view(true)` resumes execution of all the waiting opearions on `concurrent_unordered_map`.

IV. Some usage examples in pseudo code

A. User session cache

```
#include <concurrent_unordered_map>
#include <chrono>
#include <string_view>
#include <memory>

using namespace std;
void precess_user(string_view name, shared_ptr<const user_t> user);
auto get_new_user();
auto get_request();

int main() {
    concurrent_unordered_map<string_view, shared_ptr<user_t> > users;

    // single threaded fill
    read_users_from_file(users.make_unordered_map_view())
```

```
    constexpr unsigned threads_count = 10;
    while(1) {
        // concurrent work
        std::atomic<int> b{threads_count * 100500};
        thread threads[threads_count];

        for (auto& t: threads) {
            // processing users
            t = thread([&users, &b]() {
                while (--b > 0) {
                    auto [user_name, data] = co_await get_request();
                    shared_ptr<const user_t> user = users.find(user_name, shared_ptr<const user_t>{});
                    if (!user) continue;

                    precess_user(*user, data);
                }
            });
        }

        // accepting users
        auto start = std::chrono::system_clock::now();
        while (--b > 0) {
            auto [new_user_name, user] = co_await get_new_user();
            users.insert(new_user_name, user);
            if (std::chrono::duration_cast<std::chrono::milliseconds>(std::chrono::system_clock::now() - start) > 5000) {
                // debug print rough contents each 5 seconds
                users.visit_all([] (const string_view& name, const shared_ptr<user_t> user) {
                    cout << name << " " << user->status() << endl;
                });
                start = std::chrono::system_clock::now();
            }
        }

        for (auto& t: threads) {
            t.join();
        }

        // single threaded processing
        auto unsafe_users = users.make_unordered_map_view();
        count_statistics(unsafe_users);
        dump_to_file(unsafe_users);
        cleanup(unsafe_users);
    }
}
```

B. Unique events processor/events deduplicator

```
#include <concurrent_unordered_map>
#include <algorithm>

int main() {
    using namespace std;
    using event_id = ...;
    struct event_data {
        event_data(const event_data&) = delete;
        event_data& operator=(const event_data&) = delete;
        ...
    };

    concurrent_unordered_map<event_id, unique_ptr<event_data> > events;

    // Getting unique events.
    auto event_generators = get_event_generators();
    for_each(execution::par_unseq, event_generators.begin(), event_generators.end(), [&events](auto& g) {
        while (1) {
            auto [event_name, data] = co_await g.get_event();
            if (event_name.empty()) {
                break; // no more events
            }

            events.visit_or_emplace(event_name, [&data](unique_ptr<event_data>& v){
                if (v || v->priority() < data->priority()) {
                    std::swap(data, v);
                }
            }, unique_ptr<event_data>{});
        }
    });

    auto v = events.make_unordered_map_view();
    for_each(execution::par_unseq, v.begin(), v.end(), [](auto& e) {
        process(e.first, std::move(e.second));
    });
}
```

C. Gathering statistics

```
#include <concurrent_unordered_map>
#include <utility>

int main() {
    using namespace std;
    using id_t = ...;
    using use_count_t = size_t;

    concurrent_unordered_map<id_t, use_count_t> stats;

    constexpr unsigned threads_count = 10;
    thread threads[threads_count];
    for (auto& t: threads) {
```

```
        t = thread([&stats]() {
            while (1) {
                auto [id, data] = co_await get_something();
                stats.visit_or_emplace(
                    id,
                    [](auto& v){ ++v; },
                    0
                );

                precess_stuff(id, data);
            }
        });
    }

    for (auto& t: threads) {
        t.join();
    }

    process_stats(events.make_unordered_map_view());
}
```

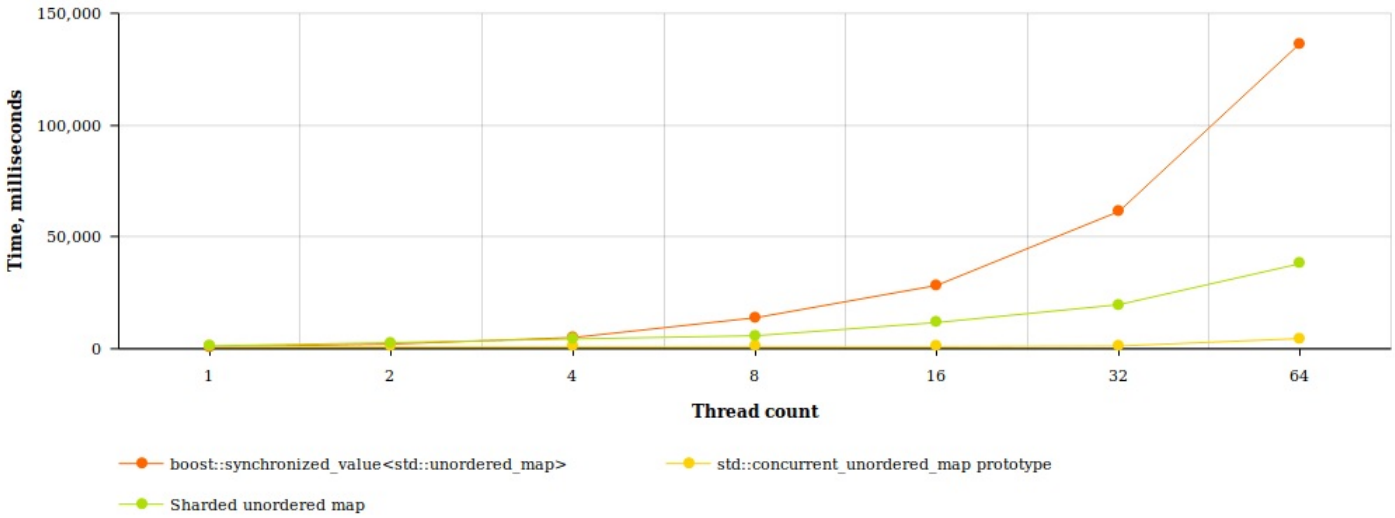
V. Performance benchmark

We've compared our implementation with a couple of simple other implementations of concurrent unordered map.
1) The simplest implementation is just a regular unordered_map with guarded by a single mutex, we've implemented mutex guard as boost::synchronized_value.
2) Many programming languages have an internal implementation of a concurrent hash map just as a fixed number of regular hash maps, each of them has it's own mutex and represents some kind of hash table shard.
3) Our reference implementation.

The test was very simple:
* All maps have string keys and int values.
* We created an appropriate number of threads that tries on each iteration to find some random value in the map and add another random value to it. If the key is absent from the map we just fill it with a random value.
* Each thread had 1000000 of iterations.
* We use a computer with 56 virtual cores & 256GB of memory for the tests.

Concurrent unordered map benchmark

Thread count	boost::synchronized_value<std::unordered_map>, ms	Sharded unordered map, ms	std::concurrent_unordered_map prototype, ms
1	837	1284	954
2	2088	2740	983
4	5018	4408	972
8	13867	5833	994
16	28314	11717	985
32	61382	19726	1224
64	136437	38021	4561



As we can see performance scales linearly when the number of threads is less than the number of cores. But when we exceed the number of cores we observe significant work time growth. We consider that the algorithm from the reference implementation is rather better than naive implementation and better than implementation from another programming languages.